

Extended Abstract

Motivation Large Language Models (LLMs) increasingly rely on step-by-step reasoning to solve complex tasks. A promising approach for enhancing rationale generation is the Self-Taught Reasoner (STaR) Zelikman et al. (2022), which iteratively generates rationales and finetunes the model on the correct reasoning traces. In doing so, STaR implicitly assumes that the answer’s correctness is a proxy of the rationale’s quality. This overlooks cases where faulty reasoning yields correct answers. To address this, we use an LLM to score rationales and use a policy gradient to directly optimize for rationale quality.

Method Our method formalizes rationale generation in LLMs as an RL problem by casting the model as a policy network that outputs rationale / answer pairs, and is trained using policy gradients. Our novel approach relies on fine-grained reward signals via a strong external verifier (GPT-4o), which scores each rationale along coherence, correctness, and relevance dimensions. These scores serve as dense reward signals that better reflect reasoning quality. To explore the impact of signal scaling on training stability and learning dynamics, we experiment with three reward normalization strategies: raw (unnormalized), mean-centered, and unit variance normalization. We find that unnormalized LLM-based rewards preserve the most signal fidelity, while normalization is more helpful in sparse binary-reward settings. For optimization, we adopt vanilla policy gradient. We found that smaller models fail to benefit from our method, so to demonstrate these benefits at scale, we experiment with larger models.

Implementation We implemented our method using Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct, training each for 2 epochs on a single NVIDIA H100 GPU. The training set consisted of 5,000 CommonsenseQA (Talmor et al., 2019) questions, with evaluation conducted on 150 validation examples. Batch sizes were set to 6 for Qwen and 3 for Llama, with a learning rate of 5e-6. We used a vanilla policy gradient loop, sampling rationale–answer pairs and computing rewards per sample. Rewards were either binary (correct/incorrect) or fine-grained, based on GPT-4o evaluations of coherence, correctness, and relevance. These were applied in raw, mean-normalized, or unit-variance-normalized forms. We built custom infrastructure for sampling, log-prob tracking, and reward integration, and used GPT-4o as an external verifier.

Results Our experiments show that rationale-aware reward shaping using LLM-based feedback significantly improves both accuracy and reasoning quality. The best performing model, Qwen2.5-1.5B-Instruct trained with unnormalized LLM-judged rewards, achieved 74.67% accuracy on CommonsenseQA, surpassing its out of the box accuracy by over 7%. This confirms that fine-grained rationale evaluation leads to more effective learning than binary correctness alone. Importantly, we found that correct answers were consistently accompanied by high-scoring rationales, while incorrect ones had lower, more varied scores, validating the LLM-as-a-judge approach. Additionally, higher BLEU scores among generated rationales aligned with better performance, suggesting that more structured and consistent rationales can serve as a proxy for both coherence and correctness. Notably, normalization harmed performance when using dense LLM-based rewards, whereas it was helpful in sparse binary reward setups. These findings reinforce that raw feedback can be more effective for guiding reasoning behavior in LLMs.

Discussion Our results show that with just a few epochs and a simple policy gradient setup, we can significantly improve LLM reasoning by using fine-grained LLM generated rationale rewards. Importantly, we note that unnormalized LLM scores outperformed normalized variants, indicating that preserving the absolute scale of the ratings provides more effective learning signals. The limitations include is that reliance on an external LLM for reward signals does not complete the loop around bootstrapping reasoning, and that there is a significant sensitivity to batch size and reward variance with this method.

Conclusion Our method demonstrates that rationale-aware reinforcement learning, using fine-grained LLM-based rewards, can significantly boost reasoning accuracy with minimal training. By leveraging finer grained rewards, we enable more effective reasoning in large models. For the future, key directions include removing dependence on external verifiers and exploring step-level rewards in the rationales.

Bootstrapping Reasoning with Reinforcement Learning

Kaiyen Yang

Department of Computer Science
Stanford University
kaieny@stanford.edu

Sanjay Nagaraj

Department of Computer Science
Stanford University
sanjay.nagaraj@stanford.edu

Jessica Chudnovsky

Department of Computer Science
Stanford University
jchud@stanford.edu

Abstract

Large Language Models (LLMs) show improved performance when generating step-by-step rationales. Existing methods like STaR reinforce correct answers even when supported by flawed reasoning, treating correctness as a proxy for rationale quality. Moreover, while STaR finetunes on correct outputs, it does not assign rewards or update gradients based on the quality of the rationales it generates, limiting its ability to directly improve reasoning. To address this, we cast STaR as a reinforcement learning problem and propose fine-grained reward signals that better reflect reasoning quality. Specifically, we prompt a strong LLM verifier (GPT-4o) to evaluate rationales along coherence, correctness, and relevance dimensions. Using policy gradients, we train models to optimize both answer accuracy and rationale quality. Our best result came from Qwen2.5-1.5B-Instruct trained with unnormalized LLM-judged rewards, achieving 74.67% on CommonsenseQA, exceeding its baseline by over 7%. The LLM verifier proved a strong signal: it correlated with correctness while providing more granular feedback. We also found that higher BLEU scores, reflecting more structured rationales, aligned with higher accuracy. Reward normalization hindered dense LLM-based signals but was helpful for sparse binary rewards. These findings highlight the promise of rationale-aware reward shaping for improving LLM performance on tasks which require reasoning.

1 Introduction

Large Language Models (LLMs) demonstrate significant performance improvements on a wide range of tasks when prompted to explicitly generate step-by-step rationales before producing a final answer (Wei et al., 2022; Nye et al., 2021; Zelikman et al., 2022). This rationale generation helps structure the model’s response and effectively encourages it to break complex problems into smaller steps.

A promising approach to enhance LLM reasoning with rationale generation is the Self-Taught Reasoner (STaR), which iteratively generates rationales and finetunes on those resulting in correct answers (Zelikman et al., 2022). In doing so, STaR implicitly assumes that the answer’s *correctness* is a proxy of the rationale’s *quality*. While largely true, this fails to account for cases where faulty reasoning nonetheless yields the correct answer.

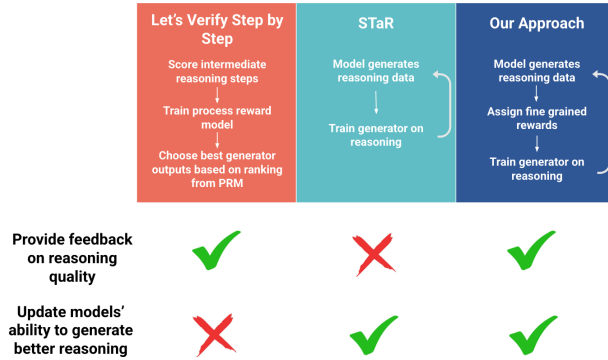


Figure 1: Motivation for Policy Gradient.

We propose refining the rationale’s reward signal to better reflect reasoning quality. Specifically, instead of providing a binary reward based on the final answer’s correctness, we aim to generate more informative, fine-grained rewards by prompting the LLM to evaluate its own rationales for consistency and coherence. We hypothesize that this may lead to not only more accurate, but also more consistent and interpretable rationales. A key step in doing this involves casting STaR as a reinforcement learning (RL) problem. Notably, this also opens the door to exploring other RL algorithms which may offer further performance gains.

2 Related Work

Our work directly builds upon STaR, a for bootstrapping reasoning in LLMs through iterative self-supervision (Zelikman et al., 2022). In STaR, the LLM: 1) generates rationales via few-shot prompting; 2) preserves those that yield correct answers; 3) retroactively constructs rationales for incorrect attempts using the correct answer as a hint; and 4) finetunes on the successful reasoning paths. While this process approximates a policy gradient objective with a binary reward signal based on answer correctness, it suffers from a key limitation: using correctness alone can reinforce flawed reasoning that happens to produce the correct answer. Our project enhances STaR by explicitly evaluating the internal coherence and logical validity of the rationales, moving beyond binary outcome supervision to introduce fine-grained reward functions. We further formalize STaR as a reinforcement learning (RL) problem, which opens the door to more robust policy optimization techniques, such as actor-critic methods.

Our work draws inspiration from Lightman et al. (2023)’s work on *process supervision*, in which step-level correctness scores are assigned within reasoning traces, in contrast to traditional *outcome supervision*, where feedback is given only for the final answer’s correctness. Although their reward model improves LLM performance on complex math problems by evaluating and ranking intermediate reasoning steps, they do not integrate this signal in a closed loop to directly train the original reasoning model. While they note that RL-based finetuning is a natural next step, our research takes this step, specifically investigating how to use such verification feedback as a signal to directly refine the base model’s reasoning capabilities through policy optimization.

By unifying the insights of STaR and process supervision, our framework casts rationale generation as an RL problem and leverages verifier feedback as a dense learning signal. This enables direct and principled optimization of the LLM’s reasoning process, bridging the gap between self-correction and reward-driven learning in LLMs.

3 Method

We cast STaR as an RL-style algorithm where the LLM is represented as a policy network $\pi_\theta(a|s)$, with

1. State s : input question
2. Action a : generated rationale and final answer $(r, \hat{y})$
3. Reward R : evaluates both the quality of reasoning in the rationale r and the correctness of the final answer $\hat{y}$

We implement the training loop using a policy gradient method described in Algorithm 1. Similar to STaR, we begin with a language model that generates rationale-answer pairs in response to a question. However, rather than relying solely on whether the final is correct (as STaR does), we treat the model as a policy network and assign a reward to each generation based on both answer correctness and the quality of the reasoning; as a baseline, we also assign binary signal for our reward mechanism. During training, we sample outputs from the model, compute their rewards, and use a policy gradient update to reinforce better reasoning behaviors. Unlike STaR, which implicitly approximates this loop through selective fine-tuning on "successful" samples, our approach formalizes it with an explicit reward-weighted update.

Algorithm 1 Policy Gradient with Granular Rationale Rewards

```

1: for  $i = 1$  to  $N$  do
2:    $B \leftarrow \emptyset$ 
3:   for each  $(x_i, y_i)$  in dataset  $D$  do
4:     Sample rationale and response:  $(r_i, \hat{y}_i) \sim \pi_\theta(x_i)$ 
5:     Evaluate rationale using language model or binary correctness:  $R_i \leftarrow \text{Score}(x_i, r_i, \hat{y}_i)$ 
6:      $B \leftarrow B \cup \{(x_i, r_i, \hat{y}_i, R_i)\}$ 
7:   end for
8:   Compute policy gradient:
9:    $\nabla J(\theta) \leftarrow \frac{1}{|B|} \sum_{(x, r, \hat{y}, R) \in B} [\nabla \log \pi_\theta(r, \hat{y} \mid x) \cdot R]$ 
10:  Update policy:
11:   $\theta \leftarrow \theta + \eta \cdot \nabla J(\theta)$ 
12: end for

```

Using Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct as the base LLMs, we explore three different methods described in Figure 2. As a baseline, we evaluate the base models Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct. Our reward paradigms include a:

- Binary reward: we assign a reward of 1 if the answer is correct, 0 otherwise, mimicking the original STaR setup. We use this reward setup as a more comprehensive baseline.
- LLM-based reward: use GPT-4o to evaluate rationales based on logical coherence, consistency, and the relevance of the rationale to the question.

First, we perform a baseline evaluation, which involves measuring the accuracy of the language model without any finetuning. Then, we introduce reinforcement learning with Binary Rewards. Finally, in our main experiment, we experiment with assigning reward signals by using a separate language model to evaluate the generated rationales. This LLM-based judge allows us to experiment with providing more informative feedback that goes beyond the final answer correctness.

In policy gradient updates, variability in reward scores can lead to significant stability issues in the policy updates. Namely, we hypothesized that the scale and distribution of reward values can significantly impact the effectiveness of the policy updates. We experimented with these three reward normalization strategies to explore their impact on training dynamics and performance.

Baseline of no normalization: Train with the rewards as is.

$$\mathcal{L}_{\text{base}} = -\frac{1}{N} \sum_{i=1}^N \text{reward}_i \cdot \log \text{prob}_i$$

In this setup, we directly apply the raw reward values to weight the log probabilities of the sampled outputs. This is the simplest approach and mirrors the classic policy gradient setup. This serves as the baseline to assess whether normalization improves performance. This method can lead to unstable updates if the reward values are noisy, highly skewed, or have inconsistent magnitude across batches.

Normalize via scaling: Scale the raw reward values to a $[0, 1]$ range.

$$\mathcal{L}_{\text{mean}} = -\frac{1}{N} \sum_{i=1}^N (\text{reward}_i - \text{baseline}) \cdot \log \text{prob}_i$$

Here, we subtract the batch mean (baseline) from each reward, effectively centering the reward distribution. We hypothesize that centering the rewards reduces variance in the gradient estimates, which can lead to more stable and efficient learning. It also can help the model focus on relative performance, where outputs that are better than average within a batch are reinforced rather than just those with absolute higher reward.

Unit variance reward: Normalizes the rewards across the batch.

$$\mathcal{L}_{\text{unit-var}} = -\frac{1}{N} \sum_{i=1}^N \left(\frac{\text{reward}_i - \text{baseline}}{\text{variance}} \right) \cdot \log \text{prob}_i$$

This method also divides by the standard deviation of the rewards in a batch, normalizing them to have zero mean and unit variance. When reward magnitudes vary significantly between batches, this normalization helps prevent large gradient spikes or vanishing updates. It standardizes the reward signal, making training more robust, especially in cases with fine-grained or inconsistent rewards like those generated by an LLM judge.

Choice of models: Our underlying hypothesis was that refining the reward signal to focus on rationale quality, rather than just final answer correctness, would lead to higher quality rationales that produce more accurate results, particularly in sufficiently expressive models. We expected these properties to emerge at scale. We found that smaller models, specifically Qwen2.5-0.5B and Llama-3.2-1B, did not benefit at all from our reinforcement learning framework, likely due to their limited capacity to represent and generalize complex reasoning patterns, which was consistent with our hypothesis. As a result, we focused our efforts on iterating with larger models such as Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct, which we believed would provide more meaningful signal in their ability to benefit from this method.

Policy Gradient: We use a vanilla policy gradient method to update the language model parameters based on reward signals derived from rationale quality and answer correctness. Policy gradient was a natural choice for this setup because it optimizes the expected return directly, rather than optimizing a value function and deriving a policy from it. This often leads to more stable convergence toward a good policy. It is a simple, scalable, and general framework that can be applied to both discrete and continuous action spaces. Compared to more complex RL methods like Q-learning or actor critic, we hypothesized that the policy gradient method would be better suited to high-dimensional, continuous action spaces like text generation. While other more advanced RL algorithms may offer sample efficiency or variance reduction, we believed that policy gradient would be sufficient for testing whether coherent reasoning signals could meaningfully guide generation.

$$\nabla J(\theta) \leftarrow \frac{1}{|B|} \sum_{(x, r, \hat{y}, R) \in B} [\nabla \log \pi_{\theta}(r, \hat{y} | x) \cdot R]$$

Prompting LLM for Meaningful Reward Signal:

To move beyond binary supervision and to obtain a more informative reward signal, we prompt a strong language model (GPT-4o) to evaluate generated rationales along multiple dimensions. Specifically, the prompt below asks the LLM to rate a rationale with respect to a given question on a scale from 1 to 5 across three axes: coherence, correctness, and relevance. The LLM returns a structured output. This approach allows us to treat the LLM as a lightweight verifier, providing fine-grained feedback that can serve as more dense reward signal in the RL loop.

We use the following prompt setup:

You are a helpful AI grader. Evaluate the following rationale for the given question.

Question: {question}
Rationale: {rationale}

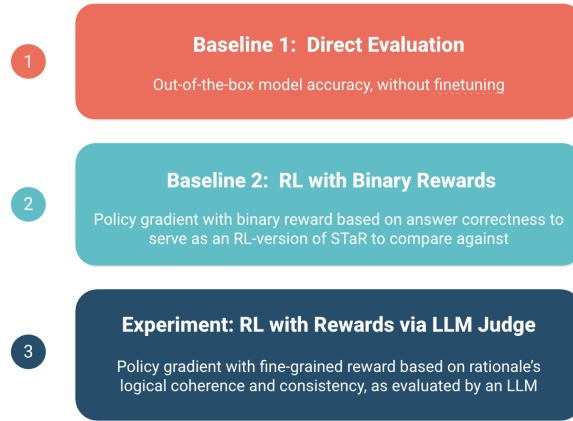


Figure 2: Method Overview

Please rate the rationale on a scale from 1 to 5 for:

1. Coherence (how logically consistent the reasoning is)
2. Correctness (whether the facts stated are accurate)
3. Relevance (whether the reasoning is focused on the question)

Output the scores as a Python list: `[coherence, correctness, relevance]`

We originally considered two alternatives for models providing more meaningful signal: (1) using the base model such as Qwen2.5-1.5B-Instruct or Llama-3.2-3B-Instruct respectively, or (2) using the newly finetuned model checkpoint from that epoch to evaluate its own rationales. However, we believed both would pose significant limitations. Using the base model risked reinforcing its initial biases or blind spots, as it lacked the reasoning quality improvements we aimed to reward. Given that the policy gradient is incredibly sensitive to variance, we opted to use a stronger external verifier which would generate more consistent rewards, and provide more stable, high-quality assessments. This setup ensured that the reward signal was maximally meaningful and stable.

4 Experimental Setup

The dataset we used for evaluation was CommonsenseQA (Talmor et al., 2019), a multiple-choice question answering dataset designed to test commonsense reasoning, including causal, spatial, etc. We used 5000 questions for training which were sampled from the training set. Due to limited computational resources and for ease of evaluations, we evaluated on 150 questions from the validation set. Experiments were run on a single H100 GPU, and due to computational limitations, we ran experiments for 2 epochs. Due to the varying sizes of the models, our batch size for the Llama-3.2-3B-Instruct was 3, and the Qwen2.5-1.5B-Instruct was 6. During training, we used a learning rate of $5e-6$. Our first baseline solely involved running inference. For the second experiment, which was binary reward training, where we cast STaR as a reinforcement learning problem via a policy gradient but with binary rewards, included three variants for training on rewards: binary rewards without normalization, binary rewards centered around the mean, and binary rewards normalized via scaling by dividing by the standard deviation (referred to as "unit variance reward"). We applied this framework for training both Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct. For the LLM as a judge method, we had a similar split in experimental setups: fine grained rewards without normalization, fine grained rewards centered around the mean, and fine grained rewards normalized via scaling. This again applied for both Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct. The binary rewards set up and out of the box model evaluations both serve as baselines, as described in Figure 2.

5 Results

Using the aforementioned setup, we had a total of 14 final experiments run, including all baselines. We include the numerical results in Table 1 for our results for the CommonsenseQA dataset.

Table 1: Comparison of models using different reward strategies and normalization techniques.

Model	Method	No normalization	Normalization (Scaling)	Unit variance reward
Qwen2.5-1.5B	Direct Evaluation	————— 0.673 [no rewards/normalization] —————		
	RL with Binary Rewards	0.25	0.7133	0.64
	RL with LLM as Judge	0.7467	0.4733	0.5933
LLama-3.2-3B	Direct Evaluation	————— 0.667 [no rewards/normalization] —————		
	RL with Binary Rewards	0.74	0.24	0.7467
	RL with LLM as Judge	0.4600	0.4067	0.5000

5.1 Quantitative Evaluation

Our experiments reveal a clear standout result: **Qwen2.5-1.5B trained with LLM-as-judge rewards and no normalization achieved the highest overall performance at 74.67%, outperforming its direct evaluation baseline by more than 7%** as well as all other configurations tested across both models. This result helps demonstrate our initial hypothesis, which is that fine-grained, rationale-based feedback updated directly into the model weights, as opposed to binary correctness, can more effectively guide LLMs toward higher-quality reasoning and consequently more accurate answers. By prompting an external verifier to assess coherence, correctness, and relevance of each generated rationale, we provided the model with a denser reward signal. We note that this reward signal was most effective in its raw, unnormalized form.

When normalization was introduced to these LLM-based rewards, performance deteriorated. With mean normalization, Qwen’s performance dropped significantly. This is likely due to the fact that centering rewards around the batch mean compresses the gradient signal and eliminates the natural scale of reward differences between high and low quality rationales. Unit variance normalization partially mitigated this, but still underperformed compared to the unnormalized setting. These results suggest that normalization tactics for Qwen are not helpful, and that high-quality, dense reward signals, contain more meaningful information. Normalization obscures the signal gained from the raw rewards generated by the external verifier. This reward signal was most effective in its raw, unnormalized form likely because the absolute scores provided by the LLM judge carried meaningful semantic distinctions. Normalizing the rewards flattens these differences, stripping away valuable info about relative quality. As a result, unnormalized rewards preserved the full expressive power of the signal, allowing the model to better reinforce high-quality reasoning patterns.

In contrast, Qwen’s performance with binary rewards exhibited the opposite trend. Without normalization, training collapsed to random, indicating that sparse feedback on final answer correctness alone was insufficient to guide effective policy updates. Applying mean normalization significantly boosted accuracy, nearly matching the direct evaluation baseline. Unit variance normalization led to a slightly lower performance, but still marked a substantial improvement over the unnormalized condition. These findings highlight that normalization is very important for sparse reward setups, where the binary signal lacks gradient richness and is prone to instability without proper scaling.

For Llama-3.2-3B, the model demonstrated different sensitivities. Its direct evaluation stood at 66.7%, roughly on par with Qwen. However, it showed strong performance with binary rewards and no normalization, reaching 74%, and increased even more with unit variance normalization. This indicates that Llama is more capable of leveraging binary signals effectively, likely due to stronger generalization from correctness-based feedback. However, applying mean normalization to binary rewards again harmed performance just as in Qwen. This highlights the consistent trend that mean normalization can destabilize training in sparse reward environments by inverting or suppressing useful signal.

In general, Llama did not benefit from LLM-as-judge rewards in any configuration. Without normalization, performance reached only 46%. Mean and unit variance normalization further reduced

this. We believe this likely can be attributed to the fact that Qwen had twice the batch size in our experimental setup relative to Llama (since Llama’s model weights take up more space given it is a bigger model), which means that the reward signal was less stable, leading to worse performance. The noisier gradient updates means that if the LLM score itself is a noisy or imperfect signal, small batches cannot smooth that out. This leads to unstable or misleading updates, which can hurt performance, which is precisely what we saw in our results. Furthermore, applying reward normalization (both mean and unit variance) assumes that we have a good estimate with of the reward distribution. But, with smaller batches, that estimate is less reliable. Normalization can amplify noise or mislead the optimization, worsening the performance.

5.2 Qualitative Analysis

Here, we analyze the results associated with the signal provided by the LLM evaluation. In our setup, we prompt the external verifier to evaluate the quality of the rationale depending on the question, and in this section, we examine the relationship between model generated answer correctness and the quality of their rationales as judged by an LLM. Specifically, we computed average rationale quality scores, based on coherence, correctness, and relevance, using an LLM grader, and compared those scores against binary correctness labels (1 for correct, 0 for incorrect) derived from the model’s original multiple-choice predictions. The resulting boxplot in Figure 3 revealed a disparity in the LLM-assessed rationale quality depending on whether the answer was correct. Rationales accompanying correct answers received consistently high scores, with a tight distribution centered near the top end of the scale (mean of the three categories: 4.5), suggesting that well-reasoned justifications often align with accurate outputs and the model is able to effectively, independently verify this. In contrast, rationales for incorrect answers displayed a much wider spread and a notably lower mean score (mean of the three categories - 3.5), with some scoring poorly across all dimensions, reflecting issues in logic, factuality, or relevance, which the external verifier is effectively able to identify.

This confirms that high-quality reasoning, as judged independently by GPT-4o, is a strong indicator of model correctness. This analysis supports the use of LLM-based evaluations for rationale grading, as can be seen in Figure 3.

Below is an example of how the external verifier evaluates and scores model generations produced by our training paradigm in our experiment for Qwen using LLM as a verifier without any normalization.

Positive Example:

- **Question:** The artist was sitting quietly pondering, then suddenly he began to paint when what struck him? Answer Choices: (a) sadness; (b) anxiety; (c) inspiration; (d) discomfort; (e) insights.
- **Rationale:** The correct answer is (c). Inspiration. Rationale: Artists often become inspired to create works of art during times of contemplation or quiet thinking, leading to the expression of ideas or emotions through their paintings. Therefore, the answer is (c). inspiring.
- **Original Score:** 1
- **LLM Evaluation:**
 - Coherence: 5
 - Correctness: 4
 - Relevance: 5
 - Raw Response: [4, 5, 5]

Negative Example:

- **Question:** James thought of criminal justice like a computer program. It needs to work right. What ideas might James not like? Answer Choices: (a) manual; (b) process information; (c) power down; (d) control model; (e) reason exists.
- **Rationale:** The answer is (manual). Therefore, the answer to this multiple-choice question is (a) manual.
- **Original Score:** 0
- **LLM Evaluation:**

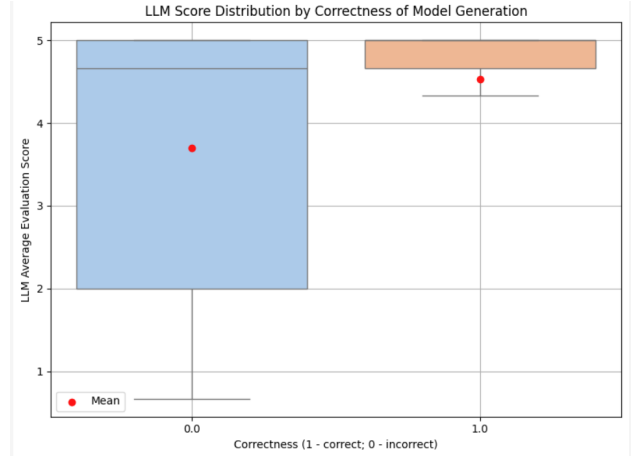


Figure 3: Qualitative analysis of LLM scoring.

- Coherence: 2
- Correctness: 1
- Relevance: 2
- Raw Response: [2, 1, 2]

In the positive example, the model correctly selects "inspiration"; and provides a well-articulated rationale that connects artistic behavior to the scenario. The quality of the explanation is reflected in the high LLM evaluation scores. This shows the LLM verifier’s ability to reward high-quality reasoning that meaningfully supports a correct answer.

In the negative example, the model selects the incorrect answer "manual" and offers a rationale that is minimal and uninformative. It is simply restating the choice without any justification. The LLM assigns low scores across all dimensions, indicating that it effectively can identify weak, unsupported reasoning and distinguish it from more substantive explanations, helping us develop meaningful feedback and reward signal for our gradient updates.

We also conducted an analysis to see how the diversity of the generations impacted the evaluation results, which can be seen in Figure 4. We note that there is a positive relationship between the BLEU and Eval scores. Meaning, as the similarity of the rationales per experimental setup increases, the likelier the eval score is to be higher. We believe this can be attributed to the fact rationales follow a similar format and language structure when effective, suggesting that the model has found a reliable reasoning pattern or strategy that it reuses, and that strategy is probably effective for solving the task. It is likely that good rationales often follow consistent patterns, hence why when the model learns to reproduce this effective structure, BLEU similarity will go up – but performance, will too, since the rationales are working better. Bleu isn’t just about diversity, but can be indicative of how focused the model is. Hence, models that generate noisy, inconsistent, or random rationales will have lower BLEU similarity, but also likely worse question-answering accuracy, because their reasoning is less grounded.

6 Discussion

A key limitation of our approach is its reliance on an external verifier (GPT-4o) to score rationales; given that our goal is to truly "bootstrap" the reasoning, we would want to be able to use the model that we fine tune to generate the next iteration of rationales. While GPT-4o yields high-quality feedback, it limits autonomy and scalability. One of the main challenges that we experienced was that we were forced to iterate with and train on a larger model. Smaller models like Qwen2.5-0.5B and Llama-3.2-1B struggled with our setup, likely due to limited capacity to learn from fine-grained rewards. We found that only larger models could benefit meaningfully from our reinforcement learning framework, but that also slowed down our iteration speed due to computational constraints. Stabilizing reward signals was another challenge. While normalization helped in sparse binary reward

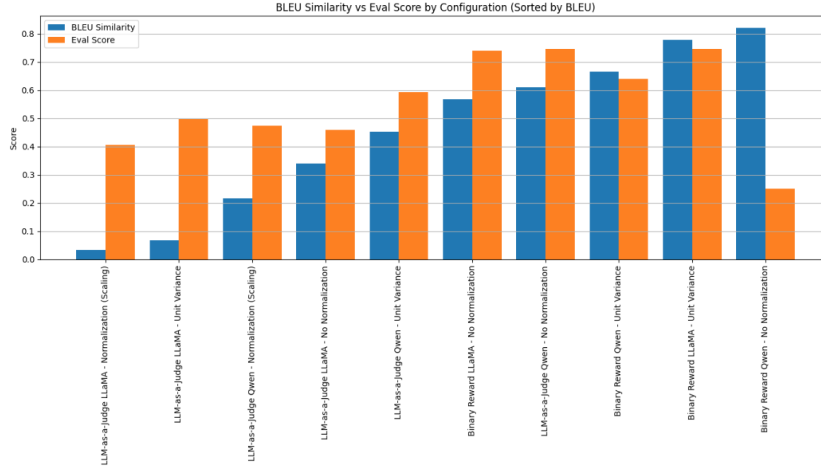


Figure 4: Bleu Scoring vs CommonsenseQA Evals

setups, it often harmed performance with dense LLM-based signals. Unnormalized rewards preserved more useful variation but made training sensitive to batch size and reward noise.

Our method shows that fine-grained, rationale-based feedback can lead to more accurate and interpretable model outputs, as evidenced by the Qwen result after 2 epochs without normalization, as evidenced by Figure 1. We demonstrate that using a strong external verifier (GPT-4o) to score rationales can provide a stable and reliable reward signal, as can be seen in Figure 3. We also show that the choice of reward normalization has a significant effect on training stability. For LLM-based rewards, raw (unnormalized) scores preserved meaningful differences between rationales and yielded the best performance, achieving experimental results that exceed both baselines. In contrast, binary rewards benefited from normalization. Another important finding was the impact of batch size. Llama, for which we used smaller batches due to memory constraints, suffered from reward signal noise and less stable policy updates, highlighting how smaller batch sizes can impair reinforcement learning in high-variance settings. Finally, our BLEU score analysis in Figure 4 revealed a correlation between higher BLEU scores and higher evaluation accuracy. This suggests that models producing more structured, standardized rationales tend to perform better. It points to the potential of BLEU or similar measures as proxy signals for rationale quality in future evaluation pipelines.

7 Conclusion

Our work demonstrates that, with the right setup, fine-grained reward signals from an external LLM verifier (GPT-4o) can meaningfully guide language models toward producing higher-quality rationales. By casting STaR (Zelikman et al., 2022) as an explicit reinforcement learning problem and updating the model via policy gradients, we showed that rationale-aware rewards outperform traditional binary correctness signals, particularly when left unnormalized.

We found that the unnormalized LLM-based rewards produced rich semantic distinctions and led to the highest accuracy which exceeded all baselines (74.67 % with Qwen), whereas normalization was still important for stabilizing binary reward training.

In addition, our findings underscore that batch size plays a critical role in training stability, especially for policy gradients. Smaller batches, as used for Llama, led to noisier updates and less stable learning, suggesting that batch-level reward smoothing is very important in such settings.

Finally, we observed that models generating more standardized rationales, as reflected by higher BLEU scores, tended to perform better. This points to structure as a useful proxy for rationale quality. We believe the most important future directions include:

More stable updates: For memory-constrained settings like we experienced for Llama, exploring less frequent gradient updates (e.g. every few batches) could improve training stability.

Step-Level Reward Granularity: Instead of assigning one reward per rationale, future work should explore scoring individual reasoning steps to better guide the model’s reasoning trajectory, and help evaluate in a more granular way what parts of rationales are helpful vs. not.

Unifying rationale generation and scoring: Rather than using a separate LLM to score the base model’s rationales, use a single model for rationale generation and scoring to more effectively realize the self-improving framework.

Off-policy RL methods: Improve sample efficiency by applying off-policy RL methods to reuse $(x, y, \hat{y}, R)$ tuples in a replay buffer, potentially upsampled for high-performing rationales.

8 Team Contributions

- **Kaien Yang:** Wrote a large portion of the codebase, including the dataset loading and batching, token logprob calculations, reward mechanism, and vanilla policy gradient updates.
- **Sanjay Nagaraj:** Built and maintained the training infrastructure, managed experiment tracking, and orchestrated training runs. Took primary responsibility for debugging issues during training, tuning hyperparameters, and ensuring overall training stability across different model and reward configurations. Worked with Jessica on LLM reward analyses and Bleu/diversity score evaluations of the rationales.
- **Jessica Chudnovsky:** Implemented the evaluation framework, including setting up baseline model inference, logging outputs, parsing results and ensuring the evaluation pipeline worked, and writing code for a reusable evaluation pipeline that supported multiple datasets. Handled running evaluations and led writing the milestone and final reports, as well as the project poster. Worked with Sanjay on LLM reward analyses and Bleu/diversity score evaluations of the rationales.

Changes from Proposal We mostly stuck to the original plan, though Sanjay and Jessica swapped roles. Since Sanjay had better access to compute, he ran most of the experiments, while Jessica focused more on logging and evaluations, which were less resource-intensive. Originally, the opposite had been planned.

References

- Jacob Andreas, Dan Klein, and Sergey Levine. 2017. Learning with latent language. *arXiv preprint arXiv:1711.00482* (2017).
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s Verify Step by Step. arXiv:2305.20050 [cs.LG] <https://arxiv.org/abs/2305.20050>
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. 2021. Show your work: Scratchpads for intermediate computation with language models. (2021).
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4149–4158. <https://doi.org/10.18653/v1/N19-1421>
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465 [cs.LG] <https://arxiv.org/abs/2203.14465>

A Additional Experiments

Originally, we intended to evaluate on the String Dataset (Andreas et al., 2017), but did not pursue this direction. String Editing, introduced in Andreas et al. (2017), involves inferring a specific text manipulation rule based on provided input-output pairs and then applying that rule to new instances. For example, the output for “warding -> warying, curved -> curvey, uranium -> uranium, drum -> yrum, pedaled -> ” would be “peyaled”, with the LLM having to infer the rule (in this case, replacing d’s with y’s). Since this dataset does not rely on any external world knowledge, it allows us to directly measure the improvement in the LLM’s reasoning capacity driven by our RL framework. However, we found that models even at the larger size that we were attempting to train really struggled with this task, and it was difficult to get any meaningful signal given that the base models were completely unable to complete this task. We attempted to filter to a smaller, easier subset of examples from this dataset, but it significantly constrained the number of examples we could train and evaluate on. The base models produced garbage generations and rationales almost all of the time.

B Implementation Details

We trained on one H100 with batch sizes 3 and 6 for the Llama-3.2-3B-Instruct and Qwen2.5-1.5B-Instruct models respectively. We implemented a full reinforcement learning training loop, including sampling rationale/answer pairs, computing log-probabilities, and applying policy gradient updates. We built infrastructure to interface with an external LLM (GPT-4o) for rationale evaluation, including structured prompting and very thorough logging and score parsing. Additional tooling included batch processing, reward normalization strategies, BLEU score computation, and logging outputs for analysis. We also created custom evaluation pipelines for baseline inference, reward-based training runs, and fine-grained comparison across model and reward configurations.